

Estimating the Longest Increasing Sequence in Polylogarithmic Time

A Deep Dive into Sublinear Additive Approximations

Based on the paper by M. Saks and C. Seshadhri

Seminar Presentation

Outline (2-3 Hour Tutorial)

- 1 Introduction and Background
- 2 Obstacles to Additive Estimations
- 3 Algorithmic Intuition
- 4 Preliminaries and Definitions
- 5 Algorithmic Building Blocks
- 6 The Basic LIS Approximation Algorithm
- 7 The Improved Algorithm

The Longest Increasing Subsequence (LIS) Problem

- **Input:** A function $f : [n] \rightarrow \mathbb{R}$, which we view as an array.
- **Increasing Subsequence:** A sequence of indices $i_1 < i_2 < \dots < i_k$ such that $f(i_1) \leq f(i_2) \leq \dots \leq f(i_k)$.
- **Goal:** Find the length of the *Longest Increasing Subsequence* (LIS).

The Longest Increasing Subsequence (LIS) Problem

- **Input:** A function $f : [n] \rightarrow \mathbb{R}$, which we view as an array.
- **Increasing Subsequence:** A sequence of indices $i_1 < i_2 < \dots < i_k$ such that $f(i_1) \leq f(i_2) \leq \dots \leq f(i_k)$.
- **Goal:** Find the length of the *Longest Increasing Subsequence* (LIS).
- **Classical Exact Algorithms:**
 - Standard Dynamic Programming: $O(n^2)$.
 - Fredman's Algorithm (1975): $O(n \log n)$ using an optimized DP.
 - Patience Sorting (Aldous and Diaconis, 1999).

Massive Datasets and Approximation

- In the era of massive datasets, $O(n \log n)$ can be too slow if n is enormous (e.g., streaming data, massive databases).
- **Question:** Can we estimate the LIS length in *sublinear* or even *polylogarithmic* time?

Massive Datasets and Approximation

- In the era of massive datasets, $O(n \log n)$ can be too slow if n is enormous (e.g., streaming data, massive databases).
- **Question:** Can we estimate the LIS length in *sublinear* or even *polylogarithmic* time?
- In sublinear algorithms, we access the array via queries to indices.
- We seek an approximate answer by observing a tiny, randomly chosen fraction of the array.

Distance to Monotonicity

- We denote the LIS length as lis_f .
- The *complement* of the LIS is the **distance to monotonicity**: $\text{loss}_f = n - \text{lis}_f$.
- This represents the minimum number of values to change to make f monotonically non-decreasing.
- **Normalized distance**: $\epsilon_f = \text{loss}_f / n$.

Previous Work in Property Testing

- Property testing distinguishes between $\text{loss}_f = 0$ (perfectly sorted) and $\text{loss}_f \geq \epsilon n$.
- Requires $O(\epsilon^{-1} \log n)$ random samples (Ergun et al. 2000, Fischer 2004).
- **Estimating Distance:**
 - Parnas, Ron, and Rubinfeld (2004) & Ailon, Chazelle, Comandur, and Liu (2004) provided algorithms to estimate loss_f .
 - Both yielded a **multiplicative** $(2 + o(1))$ -**approximation** to loss_f .
 - Running time: $(\text{loss}_f(\log n)/n)^{O(1)}$.

The Gap in Previous Knowledge

- What if lis_f is between 0 and $n/2$?
- In this case, $\text{loss}_f \geq n/2$.
- A factor-2 approximation to loss_f might return an estimate of n .
- The trivial estimate $\text{loss}_f \approx n$ implies $\text{lis}_f \approx 0$.
- **Consequence:** Previous algorithms gave an additive $n/2$ approximation for lis_f . They could not distinguish an array with LIS of size $n/4$ from an array with LIS of size $o(n)$.

Main Result: Additive Approximation (Theorem 1.1)

Theorem 1.1

There is a randomized algorithm which takes as input an array f of size n and parameter $\delta > 0$, and outputs a number est such that:

$$|\text{est} - \text{lis}_f| \leq \delta n$$

with probability at least $3/4$.

- **Running Time:** $(1/\delta)^{O(1/\delta)}(\log n)^c$, for some absolute constant c .
- For any constant δ , this is strictly *polylogarithmic* in n .

Main Result: Distance to Monotonicity (Theorem 1.2)

Theorem 1.2

Let $0 < \tau < 1$. There exists an algorithm with running time $(1/(\epsilon_f \tau))^{O(1/\tau)} (\log n)^c$ that computes ϵ such that:

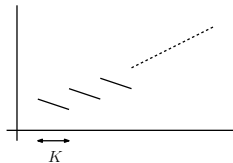
$$\epsilon_f \in [\epsilon, (1 + \tau)\epsilon]$$

with probability at least $3/4$.

- Improves the previous $(2 + o(1))$ -approximation to a $(1 + \tau)$ -approximation!

Why is this hard? Obstacle 1: Uniform Sampling

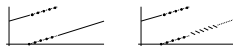
- **Naïve approach:** Take a random sample S , compute $\text{lis}_f(S)$ exactly, scale up by $n/|S|$.
- **Failure Case:**



- Array consists of blocks of size K . Within a block, items are decreasing. Blocks are globally increasing.
- $\text{lis}_f = n/K$.
- A small uniform sample will almost certainly pick at most 1 item per block, looking *completely increasing*. The estimate will be $\approx n$.

Obstacle 2: Proximity-Based Sampling

- **Previous strategy (ACCL, PRR):** Classify each index as *good* or *bad*.
- For an index i , sample points j with probability $\propto 1/|i - j|$. If i has many violations with the sample, classify it as *bad*.
- **Failure Case:**

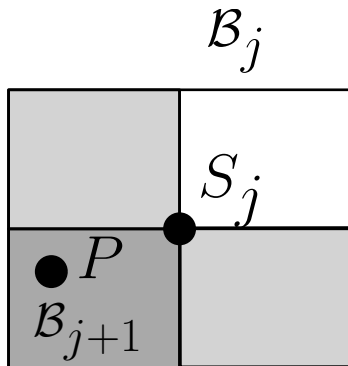


- A block's local classification may depend on small-scale properties *far away* from it. Local/proximity sampling cannot see the correct global alignment.
- Conclusion: We need a mechanism that links global structure across all scales.

An Interactive Protocol (Thought Experiment)

- Imagine an all-knowing **Prover** and a bounded **Verifier**.
- Prover claims $\text{lis}_f \geq bn$.
- **Round:**
 - 1 Verifier picks a random index $i \in [n]$. Let $F(i) = \langle i, f(i) \rangle$.
 - 2 They iteratively shrink a "bounding box". Prover offers a *splitter* S_j .
 - 3 Verifier checks if $F(i)$ is comparable to S_j . If not, reject!
 - 4 Depending on $F(i) \preceq S_j$ or $F(i) \succ S_j$, they narrow the box to the bottom-left or top-right.

Interactive Protocol Visualized



- If $F(i)$ is in the light gray regions, it's incomparable to S_j . Verifier rejects.
- If the Prover is honest (LIS is large), they can pick splitters directly from the true LIS, and the Verifier accepts with probability $\geq b$.

Simulating the Prover without an Oracle

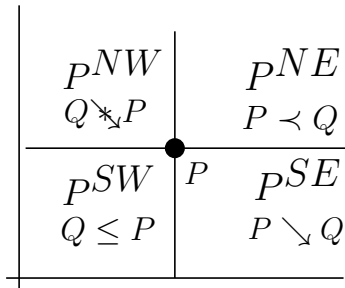
- We don't have a Prover. How do we find a good splitter algorithmically?
- A good splitter S has few *violations* (incomparable points) with the unknown LIS.
- Since we don't know the LIS, we look for a **conservative splitter**: one that has few violations with *all* points in the current box.
- If conservative splitters exist, random sampling can easily find one!

The Dichotomy: Splitter vs. Boosting

- What if there are *no* conservative splitters?
- **The Boosting Idea:**
 - If an index s causes many violations, we can use it to split the box $\mathcal{B}$ into $\mathcal{B}_L$ and $\mathcal{B}_R$.
 - Recursively approximate $\text{lis}(\mathcal{B}_L)$ and $\text{lis}(\mathcal{B}_R)$.
 - The number of points *lost* (the violations) is precisely the difference between $\text{loss}(\mathcal{B})$ and $\text{loss}(\mathcal{B}_L) + \text{loss}(\mathcal{B}_R)$.
 - If there are many violations, we significantly *boost* our approximation accuracy by dividing and conquering!
- **Dichotomy:** Either a box has a good splitter (so we can efficiently narrow our search), or it has no good splitters (meaning dividing the box yields a strictly smaller accumulated error).

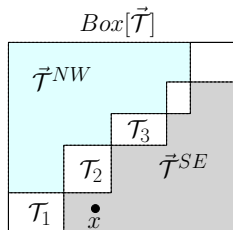
Points, Relations, and Boxes

- **Points:** Represent indices and values as $P = \langle x, y \rangle \in \mathbb{N}^2$.
- $F(x) = \langle x, f(x) \rangle$. $\mathcal{F}$ is the set of all such points.
- **Relations:**
 - $P \preceq Q$: $x(P) \leq x(Q)$ and $y(P) \leq y(Q)$.
 - $P \prec Q$: $x(P) < x(Q)$ and $y(P) \leq y(Q)$.
 - $P \searrow Q$: $x(P) < x(Q)$ and $y(P) > y(Q)$ (a *violation*).
- **Regions:** $P^{NW}, P^{SW}, P^{NE}, P^{SE}$.



Boxes and Box Chains

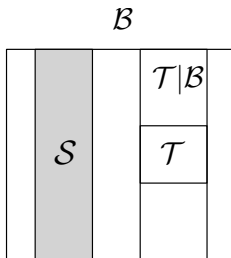
- **Box $\mathcal{B}$:** A Cartesian product of an index interval and value interval.
- **Box Sequence:** $\vec{\mathcal{B}} = (\mathcal{B}_1, \dots, \mathcal{B}_k)$ where each is strictly to the right of the previous.
- **Box Chain:** A sequence where the top-right of $\mathcal{B}_i$ is exactly the bottom-left of $\mathcal{B}_{i+1}$.



- An increasing sequence of points defines a box chain, and vice versa.

Strips and The Functions lis and loss

- **$\mathcal{B}$ -strip:** A subbox $\mathcal{S}$ of $\mathcal{B}$ that has the same vertical extent as $\mathcal{B}$.



- $\text{lis}(\mathcal{B})$: Size of the longest increasing sequence of $\mathcal{F}$ -points contained entirely in $\mathcal{B}$.
- $\text{loss}(\mathcal{B}) = |\mathcal{B} \cap \mathcal{F}| - \text{lis}(\mathcal{B})$.

The Approximation Goal

- We want an algorithm **ApproxLIS**($\mathcal{B}$) that returns an approximation.
- We measure error using two parameters: **Primary Error** (τ) and **Secondary Error** (δ).
- **Target:**

$$|\mathbf{ApproxLIS}(\mathcal{B}) - \mathbf{lis}(\mathcal{B})| \leq \tau \cdot \mathbf{loss}(\mathcal{B}) + \delta \cdot \mathbf{w}(\mathcal{B})$$

- τ will decrease with recursive depth, while δ will increase. We manage these to bound total error.

Building Block 1: Finding Splitters

- A **splitter** s divides a box $\mathcal{T}$ into a bottom-left box and a top-right box.
- We want a splitter that is:
 - ① **Balanced:** Not too close to the left or right edge (parameter ρ).
 - ② **Safe:** The number of violations inside any adjacent strip $\mathcal{S}$ is at most $L + \mu|\mathcal{F} \cap \mathcal{S}|$.
- **FindSplitter**($\mathcal{T}, \mathcal{B}, \mu, L, \rho$):
 - Randomly sample $O(\log^2 n / \rho)$ balanced points.
 - Use secondary sampling to estimate violations.
 - Return an adequate splitter if one is found.

The Dichotomy Lemma (The Theoretical Heart)

- What if **FindSplitter** fails?
- Let $\mathcal{R}$ be a box, $\vec{\mathcal{S}}$ a strip decomposition, $\vec{\mathcal{D}}$ a box chain compatible with $\vec{\mathcal{S}}$.
- Let U be the set of "unsafe" indices inside $\vec{\mathcal{D}}$.
- Let $\chi = |\mathcal{R} \cap \mathcal{F}|$, χ^{in} points inside $\vec{\mathcal{D}}$, χ^{out} points outside $\vec{\mathcal{D}}$.

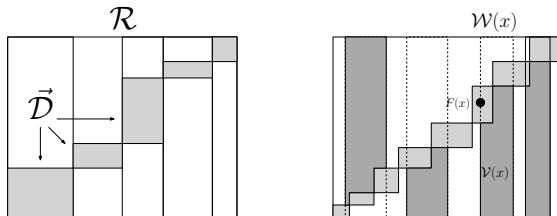
Lemma 4.6 (Dichotomy)

$$\chi^{out} \geq \frac{\mu}{1 - \mu} |U|$$

$$\chi^{in} \leq (1 - \mu)\chi + \mu|S|$$

where S is the set of safe points.

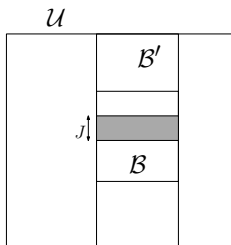
Dichotomy Lemma Visualization



- Unsafe points imply a large mass of points MUST lie outside the box chain (in regions $\mathcal{V}(x)$).
- If there are few safe points, the total points an increasing sequence can pick up is strictly bounded away from χ .

Building Block 2: Value Nets and Grids

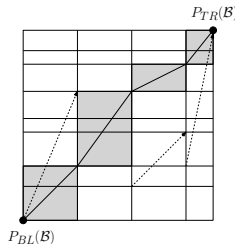
- To perform the boosting step, we partition a terminal box using a *Grid*.
- **Value Net:** A subset of values V such that any sufficiently "dense" vertical interval contains a point in V .



- **BuildGrid**($\mathcal{B}, \alpha, \xi$):
 - Creates a grid Γ with $O(1/\alpha)$ columns and $O(1/\alpha^2)$ rows.
 - Ensures fine-grained coverage of the points.

The Grid Approximation Lemma

- We form a Directed Acyclic Graph (DAG) $D(\Gamma)$ over the grid points.
- A path in $D(\Gamma)$ corresponds to a sequence of boxes.



Lemma 4.9

If Γ is an α -fine grid, then for any increasing sequence $\mathcal{L}$ in $\mathcal{B}$, there is a $D(\Gamma)$ -chain $\vec{\mathcal{D}}$ that contains all but $\alpha \cdot w(\mathcal{B})$ points of $\mathcal{L}$.

The Main Procedure: **ApproxLIS_t**($\mathcal{B}$)

- Takes a box $\mathcal{B}$ and a recursion level t .
- If $w(\mathcal{B}) = 1$, return $|\mathcal{B} \cap \mathcal{F}|$.
- **Else:**
 - Sample σ random indices from $X(\mathcal{B})$.
 - Run **Classify_t**($x, \mathcal{B}$) on each sample point x .
 - Let g be the number classified as *good*.
 - Return $g \cdot w(\mathcal{B})/\sigma$.
- The entire algorithm hinges on the **Classify** routine!

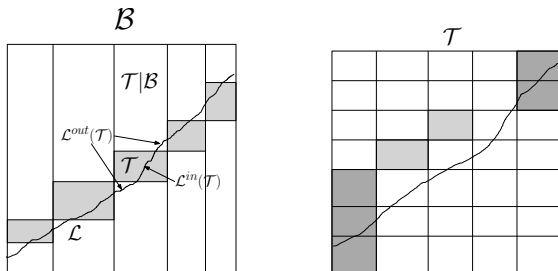
The Classify Routine: $\mathbf{Classify}_t(x, \mathcal{B})$

- Goal: Determine if x is "good" or "bad".
- If $F(x) \notin \mathcal{B}$, return *bad*.
- Base cases ($t = 0$ or $w = 1$).
- **Main Step:**
 - 1 Call $\mathbf{CriticalBox}_t(x, \mathcal{B})$ to find a subbox $\mathcal{C}$ containing x .
 - 2 Recursively call $\mathbf{Classify}_{t-1}(x, \mathcal{C})$.
 - 3 Return the result.
- What is the *Critical Box*?

Finding the Critical Box: $\text{CriticalBox}_t(x, \mathcal{B})$

- Operates in two stages:
- **Stage 1: Terminal Box**
 - Run $\text{TerminalBox}_t(x, \mathcal{B})$ to simulate the interactive protocol.
 - Keep finding splitters and shrinking the box around x until **FindSplitter** fails (or box is tiny).
 - Returns the *Terminal Box* $\mathcal{T}$.
- **Stage 2: Grid Chain / Boosting**
 - Build an α -fine grid Γ inside $\mathcal{T}$.
 - For each grid box $\mathcal{D}$, recursively evaluate $\text{ApproxLIS}_{t-1}(\mathcal{D})$ to get a weight.
 - Find the longest path in $D(\Gamma)$ to get a chain of grid boxes $\vec{\mathcal{C}}(\mathcal{T})$.
 - Return the unique box $\mathcal{C} \in \vec{\mathcal{C}}(\mathcal{T})$ containing x .

Visualizing the Terminal and Critical Chains



- Left: **TerminalBox** partitions the space into nested splits until failure. The leaves are the Terminal Chain.
- Right: Inside a terminal box, the Grid Chain solves a DP on grid sub-boxes.

Analysis: The Two Assumptions

- To analyze the randomized algorithm as deterministic, we fix random bits and assume two properties hold w.h.p.:
- **Assumption 1 (Secondary Randomness):** Every call to **FindSplitter** and **BuildGrid** is "reliable" (returns valid splitters/grids if they exist).
- **Assumption 2 (Primary Randomness):** No box-level pair $(\mathcal{B}, t)$ is "tainted".
 - Tainted means the sampling in **ApproxLIS**_{*t*} drastically failed to estimate $|Good_t(\mathcal{B})|$, or errors piled up from children.
- Both hold with probability $1 - n^{-\Omega(\log n)}$ using standard Chernoff/Hoeffding bounds.

Error Analysis: Transforming the Goal

- We want to show: $|\text{lis}(\mathcal{B}) - \mathbf{ApproxLIS}_t(\mathcal{B})| \leq \tau_t \text{loss}(\mathcal{B}) + \delta_t w(\mathcal{B})$.
- We define a discrepancy metric for a terminal box $\mathcal{T}$:

$$\Delta_t(\mathcal{T}) = (1 + \tau_t)|\mathcal{L}(\mathcal{T})| - |\text{Good}_t(\mathcal{B}) \cap \mathcal{T}| - \tau_t \beta(\mathcal{T})$$

- We bound the error by splitting it into *Primary Error* (involving τ_t, loss) and *Secondary Error* (involving δ_t , sampling bounds).

Secondary Error Terms

- We identify 5 sources of error:
 - ① ϵ_1 : Sampling error in estimating $|Good_t|$.
 - ② ϵ_2 : Cumulative error from **ApproxLIS** _{$t-1$} calls.
 - ③ ϵ_3 : Points of the true LIS lost because they fall outside the chosen grid chain.
 - ④ ϵ_4 : Points of the true LIS lost because they violate the chosen splitters.
 - ⑤ ϵ_5 : Loss accounted for by the Dichotomy Lemma inside terminal boxes.
- We prove each term is bounded by $O(\delta_w(\mathcal{B}))$.

Recurrences for τ_t and μ_t

- Setting up the algebraic inequalities requires balancing the primary error terms.
- To force the primary error to contract at each recursive level, we must set:

$$\mu_t = \frac{\tau_{t-1}}{2(1 + \tau_{t-1})}$$

- This yields the recurrence:

$$\tau_t \geq \left(\frac{2\tau_{t-1} + 2}{\tau_{t-1} + 2} \right)^2 - 1$$

- A solution to this is $\tau_t = 4/t$ and $\mu_t = 2/(t + 3)$.
- Note: As $t \rightarrow \infty$, $\tau_t \rightarrow 0$, achieving arbitrary multiplicative precision on the loss!

Bottleneck of the Basic Algorithm

- Running time of basic algorithm is $O(\log n)^{O(1/\tau)}$.
- Why? At each recursive level, we create a grid of size $\text{poly}(\log n)$ and make recursive calls.
- The depth is $t_{\max} = O(1/\tau)$.
- $(\log n)$ to the power of $(1/\tau)$ is strictly sublinear, but not purely polylogarithmic in terms of exponent independence.
- We want $(1/\epsilon)^{O(1/\tau)} \log^c n$. The exponent of $\log n$ should be a constant!

The Core Innovation: Phases in TerminalBox

- In the basic algorithm, if **FindSplitter** fails, we declare a terminal box immediately.
- In the improved algorithm, we introduce **Phases**.
- If **FindSplitter** fails, we don't stop. We increment a phase counter j , and *relax* the splitter constraints:
 - Increase the violation allowance $\gamma \rightarrow \gamma_j$.
 - Tighten the balance requirement $\rho \rightarrow \rho_j$.
 - Increase the width threshold θ .
- We only terminate when the box width drops below the threshold θ .

The Improved TerminalBox Algorithm

```
1:  $j \leftarrow 0, \theta \leftarrow \omega, \gamma \leftarrow \gamma_0, \rho \leftarrow \rho_0$ 
2: while  $w(\mathcal{T}) > \theta$  do
3:   Run FindSplitter $(\mathcal{T}, \dots, \gamma, \rho)$ 
4:   if splitter found then
5:     Split  $\mathcal{T}$  based on  $x$ 's position.
6:   else
7:     New Phase!
8:      $\theta \leftarrow \max(\theta, \gamma w(\mathcal{T})/\alpha)$ 
9:      $j \leftarrow j + 1$ 
10:     $\gamma \leftarrow \gamma_j, \rho \leftarrow \rho_j$ 
11:   end if
12: end while
```

Why do Phases Help?

- In the basic algorithm, the grid precision α was tied directly to γ , which had to be $\sim 1/\log n$.
- By progressively relaxing γ_j , we can decouple the grid precision α from $\log n$.
- We can set the grid precision α to be polynomial in an *error controller* Ψ , completely independent of n .
- This ensures the branching factor in the recursion relies only on Ψ , not on $\log n$.
- Result: Exponent of $\log n$ drops to a constant c .

Revisiting Tainted Boxes

- Since we lowered the sample size to rely on Ψ instead of $\log n$, we can no longer use a simple union bound over all possible boxes.
- A single call might fail with non-negligible probability.
- We handle this by proving that errors do not cascade catastrophically.
- **Lemma 9.7:** The probability that any box-level pair is recursively "tainted" is bounded by a carefully controlled polynomial in α .
- The expected mass of blue (tainted) sub-grids remains strictly bounded, preserving the global LIS estimate.

Final Time Complexity

- Maximum recursion depth: $t_{\max} = \lceil 4/\tau \rceil$.
- Error controller: $\Psi = \max(C, t_{\max}/\delta)$.
- Branching factor per node: $\text{poly}(\Psi)$.
- Running time: $O(\Psi^{O(t_{\max})} \log^c n)$.
- Substituting parameters: $(1/\delta\tau)^{O(1/\tau)} \log^c n$.
- The dependence on the array size n is purely $\log^c n$.

Summary of the Paper

- First polylogarithmic time algorithm for an **additive approximation** of LIS.
- Improved distance to monotonicity estimation from $(2 + o(1))$ to $(1 + \tau)$.
- **Key Techniques:**
 - Simulating Interactive Protocols without an Oracle.
 - Dichotomy between conservative splitters and approximation boosting.
 - Recursive dynamic programming on localized grids.
 - Dynamic phase relaxation to decouple structural parameters from n .
- A powerful framework for approximating DP problems in sublinear time!

Thank You!

Estimating the Longest Increasing Sequence in Polylogarithmic Time
M. Saks and C. Seshadhri